Project Structure, Workflow & Architecture

Project: Subrat Das — Portfolio + Admin CMS **Stack:** React 19 · TypeScript · Vite 7 · Tailwind CSS v4 · Framer Motion · Supabase · Vercel Functions

---

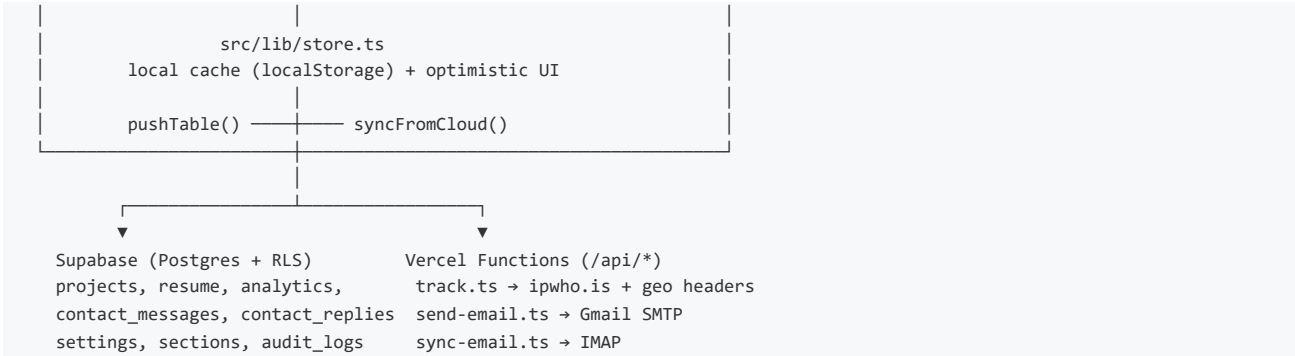
1. Folder Structure

```
My_Portfolio/
├── api/                        # Vercel serverless functions
│   ├── track.ts                # Enriched visitor tracking → analytics.meta
│   ├── send-email.ts           # Send admin replies via Gmail SMTP
│   ├── sync-email.ts           # IMAP sync of email replies into the CMS
│   └── keepalive.ts            # Supabase keepalive ping
├── docs/                       # Documentation (this PDF + README.pdf)
├── public/                     # Static assets
│   ├── certs/                  # Certificate images
│   ├── logos/                  # University / company logos
│   ├── projects/               # Project screenshots
│   └── uploads/                # Uploaded images
├── src/
│   ├── App.tsx                 # Root router (public + admin)
│   ├── main.tsx                # Entry point
│   ├── index.css               # Tailwind + theme tokens
│   ├── components/
│   │   ├── admin/              # AdminLayout, CollectionEditor, SingletonEditor, fields
│   │   ├── public/             # Portfolio sections + SectionRenderer
│   │   └── ui/                 # Reveal, Modal, Skeleton, ToastHost, ParticleField
│   ├── hooks/
│   │   └── useStore.ts         # Reactive hooks (useSyncExternalStore)
│   ├── lib/
│   │   ├── store.ts            # Reactive data layer + cloud sync + offline queue
│   │   ├── supabase.ts         # Supabase client + auth helpers
│   │   ├── types.ts            # Domain types (mirror SQL schema)
│   │   ├── seed.ts             # Initial portfolio content
│   │   ├── fieldConfigs.ts     # CMS field schemas
│   │   ├── visitor.ts          # Visitor meta parsing/formatting
│   │   ├── imageHost.ts        # Image compression + Storage upload
│   │   ├── github.ts           # GitHub REST API integration
│   │   ├── assistant.ts        # AI assistant + Web Speech API
│   │   └── emojis.ts           # Emoji picker data
│   └── pages/
│       ├── PublicSite.tsx      # Public portfolio
│       ├── AdminApp.tsx        # Admin routes
│       └── admin/              # Dashboard, Analytics, Messages, PageBuilder,
│                               # AuditLogs, Login, ForgotPassword, ResetCallback
├── supabase/                   # SQL schema + migrations
├── README.md
├── LICENSE
├── vercel.json
├── vite.config.ts
└── package.json
```

---

2. Architecture





---

3. Data Layer Workflow

1. **Local write** — mutations update the in-memory DB and `emit()` a change event, so the UI re-renders instantly.
2. **Cloud push** — `pushTable(kind, table, payload)` sends the mutation to Supabase.
3. **Failure queue** — failed writes go to `failedOps` and are retried by `retryFailedOps()`.
4. **Cloud pull** — `syncFromCloud()` fetches tables, then merges with local pending operations:
   - rows pending delete are filtered out (never resurrected)
   - pending inserts are preserved
   - pending patches keep local values
5. **Persistence** — the merged result is written to `localStorage` so refreshes are instant.
6. **Realtime** — Supabase realtime events trigger a targeted refetch (debounced).

---

4. Visitor Tracking Workflow

```
Visitor action
  │
  ├─ page_view      (PublicSite mount, once per session)
  ├─ project_view   (project modal open)          → meta = project slug
  ├─ resume_view    (resume preview open)
  └─ resume_download(resume download click)
        │
        ├─ optimistic counter bump (projects.views / resume.views / resume.downloads)
        ├─ Supabase RPC (incr_*)
        └─ POST /api/track
              ├─ IP from x-forwarded-for / x-real-ip
              ├─ geo from ipwho.is (+ Vercel edge headers fallback)
              ├─ device from UA + Client Hints
              └─ INSERT analytics { type, path, referrer, meta(JSON), created_at }
```

Admin views (`Dashboard` , `Analytics`) parse `meta` via `src/lib/visitor.ts` and show: IP · location · ISP · device · browser · source (referrer) · timestamp (DD/MM/YYYY).

Row actions: **click = highlight** (recruiter follow-up), checkbox = select, trash = delete (single or bulk).

---

5. Messaging Workflow

1. Visitor submits the contact form → row in `contact_messages` + notification email to owner.

2. Admin opens the message → reply textarea → **Send Reply**:
   - `POST /api/send-email` sends via Gmail SMTP
   - reply saved to `contact_replies` (direct insert, `add_contact_reply` RPC fallback)
3. **Sync email replies** (`/api/sync-email`) pulls replies from IMAP, deduplicates by `email_message_id`, and skips the admin's own sent copies.
4. **Open in Email** builds a Gmail compose / `mailto:` link with the visitor's original message quoted.

---

6. Git & Deployment Workflow

```
feature branch / main
        │
        ├─ npm run build      (tsc -b && vite build)
        ├─ git add / commit / push origin main
        └─ Vercel auto-deploy (production)
                ├─ static assets → CDN
                └─ /api/* → serverless functions
```

Build commands

```
npm install       # install dependencies
npm run dev       # local dev server
npm run build     # type-check + production build
npm run preview   # preview the production build
```

Environment variables

Client (`.env`): `VITE_SUPABASE_URL` , `VITE_SUPABASE_ANON_KEY` Server (Vercel): `SUPABASE_URL` , `SUPABASE_ANON_KEY` , `SMTP_*` , `IMAP_*` , `OWNER_EMAIL`

---

7. Database Migrations

File	Purpose
`schema.sql`	Full schema: tables, RLS, storage, realtime
`add-contact-replies.sql`	Reply thread table + policies
`add-reply-insert.sql`	Guaranteed reply insert / `add_contact_reply` RPC
`add-resume-views.sql`	Resume view counter
`add-counter-functions.sql`	View/download increment RPCs
`add-reset-functions.sql`	Admin reset RPCs
`add-analytics-delete.sql`	Analytics delete policy + `delete_analytics` RPC
`update-skill-categories.sql`	Framework / Other categories
`fix-resume-downloads.sql`	Fixes `incr_resume_downloads()` (missing WHERE)
`enable-realtime.sql`	Realtime publication

File	Purpose
`add-sort-order.sql`	`sort_order` column for drag ordering

---

8. Admin Screens

Route	Purpose
`/admin`	Dashboard — live stats, activity modals with delete/highlight
`/admin/profile` · `hero` · `about` · `settings`	Singleton editors
`/admin/skills` … `/admin/social-links`	Collection editors
`/admin/resume`	Resume upload (auto-updates upload date, DD/MM/YYYY)
`/admin/page-builder`	Section order, visibility, draft/publish
`/admin/messages`	Inbox, replies, email sync, CSV export
`/admin/analytics`	Traffic, referrers, visitor/project/resume activity
`/admin/audit-logs`	Every create/update/delete/login

© 2026 Subrat Das. Personal project — do not copy or redistribute without permission.